Memory Manager 아키텍처 문서

시스템 개요

Memory Manager는 운영체제의 페이징(Paging)과 가상 메모리(Virtual Memory) 개념을 AI 에이전트의 컨텍스트 관리에 적용한 시스템입니다. 3계층 계층형 메모리 아키텍처를 통해 효율적인 메모리 관리와 의미론적 검색을 제공합니다.

---

1. 시스템 아키텍처

1.1 전체 구조

```
graph LR
    subgraph "AI Agent Application"
        Agent[AI Agent]
    end

    subgraph "Memory Manager Interface"
        API[REST API]
    end

    subgraph "L1 Cache Layer"
        Redis[(Redis<br/>~1ms)]
        LRU[LRU Cache]
    end

    subgraph "L2 Vector Layer"
        Chroma[(ChromaDB<br/>~10ms)]
        Vector[Vector Search]
    end

    subgraph "L3 Storage Layer"
        Mongo[(MongoDB<br/>~50ms)]
        Persist[Persistent Storage]
    end

    subgraph "Embedding Service"
        Ollama[Ollama<br/>nomic-embed-text]
    end

    Agent -->|HTTP| API
    API --> LRU
    API --> Vector

    LRU <-->|Hot Data| Redis
    Vector <-->|Semantic| Chroma
    Persist <-->|Cold Data| Mongo

    Redis -.->|Promotion| Vector
    Vector -.->|Promotion| Persist
    Persist -.->|Eviction| Redis

    Vector <-->|Embed| Ollama

    style Redis fill:#ffa726
    style Chroma fill:#42a5f5
```

```
    style Mongo fill:#ef5350
    style Ollama fill:#66bb6a
    style LRU fill:#ab47bc
```

1.2 계층별 상세 설계

L1 캐시 계층 (Redis)

목적: 가장 빈번하게 접근하는 데이터의 고속 캐싱

특징: - 접근 시간: ~1ms - 용량: 100페이지 (기본값) - 교체 정책: LRU (Least Recently Used) - TTL: 5분 (300,000ms)

데이터 구조:

```
Key: memory:{agentId}:{key}
Value: {
  id: "uuid",
  agentId: "agent-001",
  key: "conversation:123",
  value: "serialized data",
  level: "L1_CACHE",
  accessCount: 15,
  lastAccessedAt: "2025-01-25T10:30:00Z",
  createdAt: "2025-01-25T09:00:00Z"
}
TTL: 300000ms
```

L2 벡터 계층 (ChromaDB)

목적: 의미론적 검색과 중요도 중간 데이터 저장

특징: - 접근 시간: ~10ms - 용량: 무제한 (디스크 기반) - 검색: 코사인 유사도 (Cosine Similarity) - 임베딩: nomic-embed-text (768차원)

컬렉션 구조:

```
{
  ids: ["uuid-1", "uuid-2"],
  embeddings: [[0.1, 0.2, ...], [0.3, 0.4, ...]],
  metadatas: [
    { agentId: "agent-001", key: "conversation:123", level: "L2_VECTOR" },
    { agentId: "agent-001", key: "conversation:456", level: "L2_VECTOR" }
  ],
  documents: ["User asked about AI", "Discussed ML algorithms"]
}
```

L3 저장소 계층 (MongoDB)

목적: 영구적 저장과 찾기 힘든 데이터 보관

특징: - 접근 시간: ~50ms - 용량: 무제한 - 저장 방식: 영구 저장 - Page Fault: L3 접근 시 자동 승격

도큐먼트 구조:

```
{
  _id: ObjectId("..."),
  id: "uuid",
  agentId: "agent-001",
  key: "conversation:789",
  value: "rarely accessed data",
  level: "L3_DISK",
```

```
  status: "idle",
  accessCount: 1,
  lastAccessedAt: ISODate("2025-01-25T09:00:00Z"),
  createdAt: ISODate("2025-01-25T08:00:00Z"),
  size: 1024,
  metadata: {
    type: "conversation",
    importance: "low"
  }
}
```

---

2. 메모리 관리 알고리즘

2.1 LRU (Least Recently Used) 캐시

```
graph LR
    A[Access] --> B{In Cache?}
    B -->|Yes| C[Update Access Count]
    B -->|No| D[Page Fault]
    C --> E[Move to MRU]
    D --> F[Fetch from Lower Level]
    F --> G{Cache Full?}
    G -->|Yes| H[Evict LRU]
    G -->|No| I[Insert to MRU]
    H --> I
    I --> J[Return Data]
    E --> J
```

구현:

```
class LRUCache<K, V> {
  private capacity: number;
  private cache: Map<K, V>;
  private accessOrder: DoublyLinkedList<K>;

  get(key: K): V | null {
    if (!this.cache.has(key)) {
      return null; // Cache miss
    }

    // Move to most recently used
    this.accessOrder.moveToFront(key);
    return this.cache.get(key)!;
  }

  put(key: K, value: V): void {
    if (this.cache.size >= this.capacity) {
      // Evict least recently used
      const lruKey = this.accessOrder.removeTail();
      this.cache.delete(lruKey);
    }

    this.cache.set(key, value);
    this.accessOrder.moveToFront(key);
  }
}
```

2.2 페이지 부족(Page Fault) 처리

```
sequenceDiagram
    participant Client
    participant L1 as L1 Cache
    participant L2 as L2 Vector
    participant L3 as L3 Storage
    participant Embed as Embedding

    Client->>L1: GET(key)
    L1-->>Client: Cache Miss (Page Fault)

    Client->>L2: GET(key)
    alt Found in L2
        L2-->>Client: Return Data
        Client->>L1: PUT(key) [Promotion]
        L1-->>Client: Stored in L1
    else Not in L2
        L2-->>Client: Cache Miss
        Client->>L3: GET(key)
        L3-->>Client: Return Data
        Client->>Embed: Generate Embedding
        Embed-->>Client: Vector
        Client->>L2: PUT(key, vector) [Promotion]
        Client->>L1: PUT(key) [Promotion]
    end
```

페이지 부족 해결 단계:

1. **L1 Cache Miss**: L2에서 검색
2. **L2 Vector Miss**: L3에서 검색
3. **L3 Storage Hit**: 데이터 반환
4. 임베딩 생성: Ollama로 벡터 생성
5. **L2**에 저장: 벡터와 함께 저장
6. **L1**에 승격: 자주 사용될 것으로 예상

2.3 자동 승격/강등 (Auto Promotion/Demotion)

```
stateDiagram-v2
    [*] --> L3: Initial Write
    L3 --> L2: Access Count > Threshold
    L2 --> L1: Frequent Access
    L1 --> L2: Access Count Decay
    L2 --> L3: Rarely Used
    L1 --> [*]: LRU Eviction
    L2 --> [*]: Capacity Full

    note right of L1
        Hot Data
        Fast Access (~1ms)
        Limited Capacity
    end note

    note right of L2
        Warm Data
        Semantic Search
        Medium Access (~10ms)
    end note
```

```
    note right of L3
        Cold Data
        Persistent Storage
        Slow Access (~50ms)
    end note
```

승격 조건: - L3 → L2: `accessCount > 5` - L2 → L1: `accessCount > 10` AND `recentAccesses < 1hour`

강등 조건: - L1 → L2: `accessCount < 3` AND `timeSinceLastAccess > 10min` - L2 → L3: `accessCount < 1` AND `timeSinceLastAccess > 1hour`

---

3. 의미론적 검색 (Semantic Search)

3.1 임베딩 파이프라인

```
graph LR
    Input[Input Text] --> Ollama[Ollama<br/>nomic-embed-text]
    Ollama --> Vector[768-dim Vector]
    Vector --> Chroma[ChromaDB<br/>Vector Store]
    Chroma --> Search[Cosine Similarity<br/>Search]
    Search --> Results[Top K Results]
```

임베딩 생성:

```typescript
class OllamaEmbeddingService {
  async embed(text: string): Promise<number[]> {
    const response = await fetch(`${this.baseURL}/api/embeddings`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        model: 'nomic-embed-text',
        prompt: text
      })
    });

    const data = await response.json();
    return data.embedding; // [0.1, 0.2, ..., 0.768] (768 dimensions)
  }
}
```

3.2 유사도 검색 알고리즘

코사인 유사도:

```
similarity(A, B) = (A · B) / (||A|| * ||B||)

where:
  A · B = Σ(Ai * Bi)  (dot product)
  ||A|| = √(Σ(Ai²))   (magnitude)
```

검색 프로세스: 1. 쿼리 텍스트를 임베딩으로 변환 2. ChromaDB에서 유사한 벡터 검색 3. 상위 K개 결과 반환 (기본값: 5) 4. 유사도 점수 필터링 (기본값: > 0.5)

---

4. 데이터 흐름

4.1 읽기(READ) 작업 흐름

```
sequenceDiagram
    participant A as AI Agent
    participant M as Memory Manager
    participant L1 as L1 Cache
    participant L2 as L2 Vector
    participant L3 as L3 Storage

    A->>M: GET(agentId, key)
    M->>L1: Check Cache

    alt Hit in L1
        L1-->>M: Data (~1ms)
        M->>L1: Update Access Count
        L1-->>M: OK
        M-->>A: Result (level=L1, pageFault=false)
    else Miss in L1
        M->>L2: Search Vector DB

        alt Hit in L2
            L2-->>M: Data (~10ms)
            M->>L1: Store in L1 [Promotion]
            M-->>A: Result (level=L2, pageFault=false)
        else Miss in L2
            M->>L3: Query MongoDB
            L3-->>M: Data (~50ms)
            M->>Ollama: Generate Embedding
            Ollama-->>M: Vector
            M->>L2: Store with Vector [Promotion]
            M->>L1: Store in L1 [Promotion]
            M-->>A: Result (level=L3, pageFault=true)
        end
    end
```

4.2 쓰기(WRITE) 작업 흐름

```
sequenceDiagram
    participant A as AI Agent
    participant M as Memory Manager
    participant L1 as L1 Cache
    participant L2 as L2 Vector
    participant L3 as L3 Storage
    participant O as Ollama

    A->>M: PUT(agentId, key, value)
    M->>O: Generate Embedding
    O-->>M: Vector

    par Parallel Write
        M->>L1: Store in Cache
        and
        M->>L2: Store with Vector
        and
        M->>L3: Store Persistently
    end
```

```
    L1-->>M: OK
    L2-->>M: OK
    L3-->>M: OK

    M-->>A: Success (level=L1)
```

---

5. 도메인 모델

5.1 핵심 도메인

```
// 메모리 페이지 (기본 단위)
interface MemoryPage {
  id: string;                     // UUID
  agentId: string;                // 에이전트 ID (격리)
  key: string;                    // 페이지 식별자
  value: string;                  // 직렬화된 데이터
  embedding?: number[];           // 벡터 임베딩
  level: MemoryLevel;             // L1_CACHE | L2_VECTOR | L3_DISK
  status: PageStatus;             // active | idle | swapped_out | evicted
  accessCount: number;            // LRU 추적
  lastAccessedAt: Date;           // 마지막 접근 시간
  createdAt: Date;                // 생성 시간
  size: number;                   // 페이지 크기 (bytes)
  metadata?: Record<string, any>; // 추가 메타데이터
}

// 페이지 테이블 엔트리 (주소 변환)
interface PageTableEntry {
  pageNumber: number;             // 논리 페이지 번호
  frameNumber?: number;           // 물리 프레임 번호
  level: MemoryLevel;             // 현재 위치
  present: boolean;               // 메모리에 있는지?
  referenced: boolean;            // 최근 참조 여부 (LRU)
  modified: boolean;              // 수정 여부 (Dirty bit)
  lastAccessTime: Date;           // 마지막 접근 시간
}

// 메모리 관리자 통계
interface MemoryManagerStats {
  l1Size: number;                 // L1 현재 크기
  l1Capacity: number;             // L1 용량
  l2Size: number;                 // L2 현재 크기
  l3Size: number;                 // L3 현재 크기
  totalAccesses: number;          // 전체 접근 횟수
  pageFaults: number;             // 페이지 부족 횟수
  hitRate: number;                // 캐시 적중률 (0-1)
  averageAccessTime: number;      // 평균 접근 시간 (ms)
  evictions: number;              // 교체 횟수
  promotions: number;             // 승격 횟수
  demotions: number;              // 강등 횟수
}
```

5.2 계층형 메모리 관리자

```
class HierarchicalMemoryManager {
  private l1Cache: LRUCache<string, MemoryPage>;
```

```
private l2Vector: ChromaDBVectorStore;
private l3Storage: MongoDBPageStore;
private embeddingService: OllamaEmbeddingService;

async get(agentId: string, key: string): Promise<MemoryAccessResponse> {
  const startTime = Date.now();

  // L1 캐시 확인
  const l1Hit = await this.l1Cache.get(this.makeKey(agentId, key));
  if (l1Hit) {
    return {
      success: true,
      data: l1Hit.value,
      level: MemoryLevel.L1_CACHE,
      accessTime: Date.now() - startTime,
      pageFault: false
    };
  }

  // L2 벡터 검색
  const l2Hit = await this.l2Vector.get(agentId, key);
  if (l2Hit) {
    // L1로 승격
    await this.l1Cache.put(this.makeKey(agentId, key), l2Hit);
    return {
      success: true,
      data: l2Hit.value,
      level: MemoryLevel.L2_VECTOR,
      accessTime: Date.now() - startTime,
      pageFault: false
    };
  }

  // L3 저장소 검색 (Page Fault)
  const l3Hit = await this.l3Storage.get(agentId, key);
  if (l3Hit) {
    // 임베딩 생성 후 L2, L1로 승격
    const embedding = await this.embeddingService.embed(l3Hit.value);
    l3Hit.embedding = embedding;

    await this.l2Vector.put(agentId, key, l3Hit);
    await this.l1Cache.put(this.makeKey(agentId, key), l3Hit);

    return {
      success: true,
      data: l3Hit.value,
      level: MemoryLevel.L3_DISK,
      accessTime: Date.now() - startTime,
      pageFault: true
    };
  }

  return {
    success: false,
    message: 'Key not found'
  };
```

```
  }

  async put(agentId: string, key: string, value: string):
Promise<MemoryAccessResponse> {
    const startTime = Date.now();

    // 임베딩 생성
    const embedding = await this.embeddingService.embed(value);

    const page: MemoryPage = {
      id: this.generateUUID(),
      agentId,
      key,
      value,
      embedding,
      level: MemoryLevel.L1_CACHE,
      status: PageStatus.ACTIVE,
      accessCount: 1,
      lastAccessedAt: new Date(),
      createdAt: new Date(),
      size: Buffer.byteLength(value, 'utf8')
    };

    // 병렬 쓰기
    await Promise.all([
      this.l1Cache.put(this.makeKey(agentId, key), page),
      this.l2Vector.put(agentId, key, page),
      this.l3Storage.put(agentId, key, page)
    ]);

    return {
      success: true,
      level: MemoryLevel.L1_CACHE,
      accessTime: Date.now() - startTime,
      pageFault: false
    };
  }
}
```

---

6. 성능 최적화

6.1 캐시 전략

다단계 캐싱: - L1: 자주 접근하는 데이터 (1ms) - L2: 의미론적 검색 가능한 데이터 (10ms) - L3: 영구 저장소 (50ms)

캐시 워밍:

```
async function warmCache(agentId: string) {
  const hotKeys = await identifyHotKeys(agentId);
  for (const key of hotKeys) {
    await memoryManager.get(agentId, key); // L1로 로드
  }
}
```

6.2 배치 처리

```
async function batchGet(agentId: string, keys: string[]): Promise<MemoryPage[]> {
  const promises = keys.map(key => memoryManager.get(agentId, key));
  return await Promise.all(promises);
}
```

6.3 압축

큰 데이터는 압축하여 저장:

```
import { gzip, ungzip } from 'zlib';

async function compress(value: string): Promise<Buffer> {
  return await gzip(Buffer.from(value));
}

async function decompress(buffer: Buffer): Promise<string> {
  const decompressed = await ungzip(buffer);
  return decompressed.toString('utf-8');
}
```

---

7. 동시성 제어

7.1 낙관적 잠금 (Optimistic Locking)

```
interface VersionedPage extends MemoryPage {
  version: number;
}

async putWithVersion(
  agentId: string,
  key: string,
  value: string,
  expectedVersion: number
): Promise<boolean> {
  const current = await this.get(agentId, key);

  if (current.version !== expectedVersion) {
    throw new Error('Conflict: Page was modified');
  }

  const newPage = { ...current, value, version: current.version + 1 };
  await this.put(agentId, key, newPage.value);

  return true;
}
```

7.2 분산 잠금 (Redlock)

```
import Redlock from 'redlock';

const lock = await redlock.acquire([`locks:${agentId}:${key}`], 1000);

try {
  await memoryManager.put(agentId, key, value);
} finally {
  await lock.release();
}
```

8. 모니터링 및 메트릭

8.1 핵심 메트릭

메트릭	설명	목표치
hitRate	L1 + L2 적중률	> 80%
pageFaultRate	L3 접근 비율	< 20%
avgAccessTime	평균 접근 시간	< 10ms
l1Utilization	L1 사용률	70-90%
evictionRate	교체 비율	< 10%

8.2 메트릭 수집

```
class MetricsCollector {
  private prometheus = require('prom-client');

  constructor() {
    this.hitRate = new this.prometheus.Gauge({
      name: 'memory_hit_rate',
      help: 'Cache hit rate'
    });

    this.accessTime = new this.prometheus.Histogram({
      name: 'memory_access_time_ms',
      help: 'Memory access time in milliseconds',
      buckets: [1, 5, 10, 25, 50, 100]
    });
  }

  recordAccess(level: MemoryLevel, accessTime: number): void {
    this.accessTime.observe(accessTime);

    if (level !== MemoryLevel.L3_DISK) {
      this.hitRate.inc();
    }
  }
}
```

9. 배포 아키텍처

9.1 단일 노드 배포

```
graph LR
    subgraph "Single Server"
        App[Memory Manager App]
        Redis[(Redis)]
        Chroma[(ChromaDB)]
        Mongo[(MongoDB)]
        Ollama[Ollama]
    end

    App --> Redis
    App --> Chroma
```

```
    App --> Mongo
    Chroma --> Ollama
```

9.2 분산 배포

```
graph LR
    subgraph "Load Balancer"
        LB[HAProxy/Nginx]
    end

    subgraph "App Servers"
        App1[App 1]
        App2[App 2]
        App3[App 3]
    end

    subgraph "Shared Storage"
        RedisCluster[(Redis Cluster)]
        ChromaCluster[(ChromaDB Cluster)]
        MongoReplicaSet[(MongoDB Replica Set)]
    end

    subgraph "Embedding Service"
        Ollama1[Ollama 1]
        Ollama2[Ollama 2]
    end

    LB --> App1
    LB --> App2
    LB --> App3

    App1 --> RedisCluster
    App2 --> RedisCluster
    App3 --> RedisCluster

    App1 --> ChromaCluster
    App2 --> ChromaCluster
    App3 --> ChromaCluster

    App1 --> MongoReplicaSet
    App2 --> MongoReplicaSet
    App3 --> MongoReplicaSet

    ChromaCluster --> Ollama1
    ChromaCluster --> Ollama2
```

---

10. OS 개념 매핑 상세

10.1 페이징 (Paging)

가상 메모리 시스템: - 논리 주소: `agentId:key` 형식의 키 - 물리 주소: 실제 저장 위치 (Redis, ChromaDB, MongoDB) - 페이지 크기: 가변 (JSON 직렬화 크기)

페이지 테이블:

```
interface PageTable {
  entries: Map<string, PageTableEntry>;
```

```
  translate(logicalAddress: string): PhysicalAddress {
    const entry = this.entries.get(logicalAddress);

    if (!entry || !entry.present) {
      throw new PageFaultException(logicalAddress);
    }

    return {
      level: entry.level,
      frameNumber: entry.frameNumber
    };
  }
}
```

10.2 스래싱(Thrashing) 방지

스래싱 감지:

```
class ThrashingDetector {
  private recentPageFaults: number[] = [];

  detect(): boolean {
    const recentFaults = this.recentPageFaults.slice(-100);
    const faultRate = recentFaults.filter(f => f === 1).length / recentFaults.length;

    return faultRate > 0.5; // 50% 이상이 Page Fault이면 Thrashing
  }
}
```

해결책: 1. 워킹 세트 모델: 자주 접근하는 페이지 집합 유지 2. **L1** 용량 증설: 더 많은 핫 데이터 유지 3. 접근 패턴 분석: 예측적 프리패칭

---

11. 기술 스택

계층	기술	버전	용도
언어	TypeScript	5.9	타입 안전성
런타임	Node.js	20 LTS	서버 런타임
웹 프레임워크	Express.js	4.18	REST API
L1 캐시	Redis	7.2	고속 캐싱
L2 벡터 DB	ChromaDB	0.4+	의미론적 검색
L3 저장소	MongoDB	7.0	영구 저장
임베딩	Ollama	Latest	로컬 임베딩
검증	Zod	3.22	스키마 검증
테스트	Jest	29.7	단위/통합 테스트

---

12. 참고 자료

- OS Concepts: Paging and Virtual Memory
- ChromaDB Documentation
- Redis Documentation
- MongoDB Documentation
- Ollama Documentation